

PACE Solver Description: Δ Search

Arnav Bajjuri ✉

Foothill High School, Pleasanton, CA, USA

Farhana Akter Tumpa ✉ 

Department of Computer Science, University of California, Riverside, CA, USA

Rebin Silva Valan Arasu ✉ 

Department of Computer Science, University of California, Riverside, CA, USA

Rajiv Gupta ✉ 

Department of Computer Science, University of California, Riverside, CA, USA

Abstract

We describe our heuristic solver, Δ Search, for the *Maximum Agreement Forest* (MAF) problem submitted to the PACE 2026 challenge. Given two rooted binary phylogenetic trees over a defined leaf set, the MAF problem asks for an agreement forest with the fewest components and this problem is NP-hard to compute. Our solver first reduces the instance with the kernelization rules [3], and then runs three algorithms sequentially: (i) a duality-based 2-approximation [4] that supplies a fast initial solution, (ii) our Δ Search heuristic that repeatedly refines a feasible forest by a binary search-like method over the edge set, and (iii) a randomized bounded-search approximation [2] that keeps refining the forest until the time limit. The largest agreement forest found within the time budget is returned. Δ Search casts MAF as a generic subgraph extraction problem which needs only a feasibility test and an objective, so this search proceeds without any problem-specific tuning.

2012 ACM Subject Classification Theory of computation $\rightarrow$ Fixed parameter tractability; Theory of computation $\rightarrow$ Graph algorithms analysis; Theory of computation $\rightarrow$ Approximation algorithms analysis

Keywords and phrases Maximum Agreement Forest, rSPR distance, phylogenetics, heuristic search, Δ Search, kernelization

1 Introduction

Two genes of the same set of species can produce two different rooted binary phylogenetic trees, and their dissimilarity is commonly measured by the rooted Subtree Prune and Regraft (rSPR) distance. This distance equals one less than the number of components of a *Maximum Agreement Forest* (MAF) of the two trees, the largest common forest obtained by deleting edges from both trees and contracting vertices of degree two. Computing it is NP-hard, so kernelization, approximation algorithms and heuristic-driven exponential time searches have been developed [6, 2, 4].

We document our submission to the *Heuristic* track of PACE 2026 [5]. The Heuristic track asks for a large (in number of edges) agreement forest of two rooted binary trees within a short time limit. Our solver employs three different algorithms to achieve this. The solver starts with the algorithm from [4] which provides a 2-approximation solution which is minimum solution quality required by the challenge. Our solver then employs Δ Search which is a general heuristic framework for optimal subgraph extraction that we developed. Δ Search is an algorithm that can find near optimal subgraphs for graph extraction problems given only the feasibility constraint and the optimization objective. Hence, Δ Search is instantiated with an agreement forest checker and agreement forest size calculator. After running Δ Search for a fixed time budget, we run the bounded-search approximation of [2], with randomized restarts, until the time limit. The two algorithms [4, 2] other than Δ Search and the reduction techniques [3] are reimplemented from published papers.

2 Preliminaries

Let T_1 and T_2 be rooted binary trees on the leaf set X with $|X| = n$. A forest of rooted binary trees can be obtained by deleting edges from a rooted binary tree and suppressing the resulting vertices with only one child. An *agreement forest* of T_1 and T_2 is a forest that can be arrived at from both T_1 and T_2 in the same way. The MAF problem seeks an agreement forest with the least number of components k . In the challenge, the input trees are provided in Newick notation and the output must list the components of the agreement forest as trees in Newick notation.

3 Kernelization

Before the solver starts with the search, it exhaustively applies the reduction rules from [3] which preserve k . The reduction rules are:

Subtree reduction. A maximal subtree that is identical in T_1 and T_2 is replaced in both trees by a single new leaf, since it never needs to be broken by an optimal solution.

Chain reduction. A common chain (a maximal sequence of leaves attached along a path in the same order in both trees) is truncated to a short canonical stub, capping the chain length that the search must consider.

(3, 2)-chain reduction. The strengthened rule from [3] that handles the short chain and cherry configurations left after the two rules above, tightening the kernel.

Rules are applied until a fixed point is reached. The reduction changes are pushed onto an undo stack so any solution found can be translated to a solution for the original instance by undoing each reduction in reverse.

4 Solver Architecture

After kernelization the reduced instance is handed to the combination of three algorithms. Each algorithm reports every feasible forest it finds through a shared solution manager that keeps the one with the fewest components. The solution manager prints the best solution found when the time limit is reached. The algorithms are run in order so that high quality bounds are available early and the remaining time budget is spent on finding better solutions.

4.1 2-approximation warm start

We first run the combinatorial duality based 2-approximation [4]. This algorithm maintains a red/blue/white vertex colouring which defines partitions over the trees. At each iteration, the algorithm processes the lowest node that makes the partitions an infeasible solution and fixes it. When the entire tree is processed, there will exist no lowest node of infeasibility and thus the solution will be a valid solution. This gives a quick solution that is 2-approximate.

4.2 Δ Search for MAF

Our general subgraph extraction algorithm, Δ Search [1], selects a *feasible* subset of graph elements and optimises it based on an objective. Δ Search explores the search space in a binary-search like manner that calls the feasibility and objective functions at most $O(m)$ times where m is the number of edges in the original graph.

Algorithm 1 Δ Search refinement for Maximum Agreement Forest.

Input: rooted binary trees T_1, T_2 ; leaf set X
Function: $isValidMAF(F)$: **True** iff $T_1[F]$ can agree with some subgraph of T_2
Function: $NumComponents(F)$: number of components in $T_1[F]$ induced by X

```

1: procedure DELTASEARCH ( $T_1, T_2, X$ )
2:    $best \leftarrow X$ ;  $bestCost \leftarrow |X|$  ▷ Fully cut forest
3:    $pool \leftarrow E(T_1)$ 
4:   while not timeout do
5:      $F \leftarrow \emptyset$ ;  $cost \leftarrow |X|$  ▷ Start fully cut
6:      $n \leftarrow 1$ 
7:     while  $n < |pool|$  do
8:        $n \leftarrow \min(2n, |pool|)$ 
9:        $B[0..n-1] \leftarrow$  exclusive contiguous partitions of  $pool$ 
10:      for  $i \leftarrow n-1$  downto 0 do
11:        if  $NumComponents(F \cup B[i]) < cost$  and  $isValidMAF(F \cup B[i])$  then
12:           $F \leftarrow F \cup B[i]$ ;  $cost \leftarrow NumComponents(F)$ 
13:           $pool \leftarrow pool \setminus B[i]$  ▷ Commit block
14:          if  $cost < bestCost$  then
15:             $best \leftarrow F$ ;  $bestCost \leftarrow cost$ 
16:          end if
17:        end if
18:      end for
19:    end while
20:     $pool \leftarrow E(T_1)$ ; SHUFFLE( $pool$ ) ▷ Reset, randomise
21:  end while
22:  return  $best$ 
23: end procedure

```

84 We instantiate Δ Search with the edges of T_1 and it starts with a feasible solution of
 85 deleting all the edges. Then, it adds back the edges as subsets with decreasing sizes starting
 86 from subsets of size half the total number of edges. This adaptive granularity search finds
 87 maximal components much faster than greedy algorithms and with more sensitivity to
 88 ordering which can be exploited by running it with different ordering multiple times to get
 89 different solutions.

90 Δ Search runs for multiple rounds until its time budget is exhausted. Each round starts
 91 with a forest of nodes with no edges $F = \emptyset$ and iteratively edges are added to this initial
 92 solution. In the first iteration, the edge set is divided into two (line 9) and the search tries
 93 to add each of these subsets to the solution. In the next iteration, all the remaining edges
 94 are divided into four (line 9) and the search tries to add each of these new subsets to the
 95 solution. Thus at each iteration, the granularity of edge additions becomes finer and finer.
 96 The edge addition proceeds only when the new forest agrees with the second tree and if
 97 it has strictly fewer components than existing solution in the round (line 13). The search
 98 relies only on two MAF-specific functions: $isValidMAF$, which tests whether the given
 99 edge subset of T_1 induces an agreement forest of T_2 , and $NumComponents$, the objective
 100 we try to minimise. Both of them are derived directly from the definition of MAF problem.
 101 Δ Search also requires no tuning and the pool of edges is reset and shuffled randomly for
 102 every round with no problem specific guidance.

103 4.3 Randomized Approximation Search

104 The remaining time budget is allocated to the approximation algorithm of Chen et al. [2].
 105 It contracts common cherries into a compact kernel, then runs a bounded descent that, for
 106 each unresolved sibling pair, either contracts it, cuts one side, or cuts the pendants along
 107 the connecting path when few enough. Randomized restarts vary the threshold and the
 108 cut choices, so each descent yields a different agreement forest. The forest with the fewest
 109 components is kept.

110 5 Implementation and Correctness

111 The solver is a single C++17 file (`maf_combined.cpp`) with no external dependencies. The
 112 solver comes with a Makefile and can be built with the `make` command. It reads from
 113 standard input and writes the forest to standard output in the challenge format. Trees are
 114 stored in flat index-based arrays with reusable scratch buffers so that memory allocations are
 115 not required during the Δ Search rounds. A watchdog thread enforces the time budget for
 116 the Δ Search and a signal handler is responsible for writing the current solution on a `SIGTERM`
 117 signal. The correctness of kernelization, the 2-approximation and the approximation of Chen
 118 et al. follows from their respective papers [3, 4, 2] and the correctness of Δ Search follows
 119 from the fact that it uses a feasibility function to set the solutions.

120 — References —

- 121 1 Rebin Silva Valan Arasu and Rajiv Gupta. Solving subgraph extraction problems using
 122 Δ Search, 2026. URL: <https://arxiv.org/abs/2606.13834>, `arXiv:2606.13834`.
- 123 2 Zhi-Zhong Chen, Youta Harada, Yuna Nakamura, and Lusheng Wang. Faster exact computa-
 124 tion of rspr distance via better approximation. *IEEE/ACM Transactions on Computational*
 125 *Biology and Bioinformatics*, 17(3):916–929, 2018.
- 126 3 Steven Kelk, Simone Linz, and Ruben Meuwese. Cyclic generators and an improved linear
 127 kernel for the rooted subtree prune and regraft distance. *Information Processing Letters*,
 128 180:106336, 2023.
- 129 4 Neil Olver, Frans Schalekamp, Suzanne van Der Ster, Leen Stougie, and Anke van Zuylen.
 130 A duality based 2-approximation algorithm for maximum agreement forest. *Mathematical*
 131 *Programming*, 198(1):811, 2022.
- 132 5 PACE Challenge. PACE 2026: Maximum Agreement Forest. [https://pacechallenge.org/](https://pacechallenge.org/2026/)
 133 `2026/`, 2026. Accessed: 2026-07-07.
- 134 6 Chris Whidden, Robert G Beiko, and Norbert Zeh. Fixed-parameter algorithms for maximum
 135 agreement forests. *SIAM Journal on Computing*, 42(4):1431–1466, 2013.